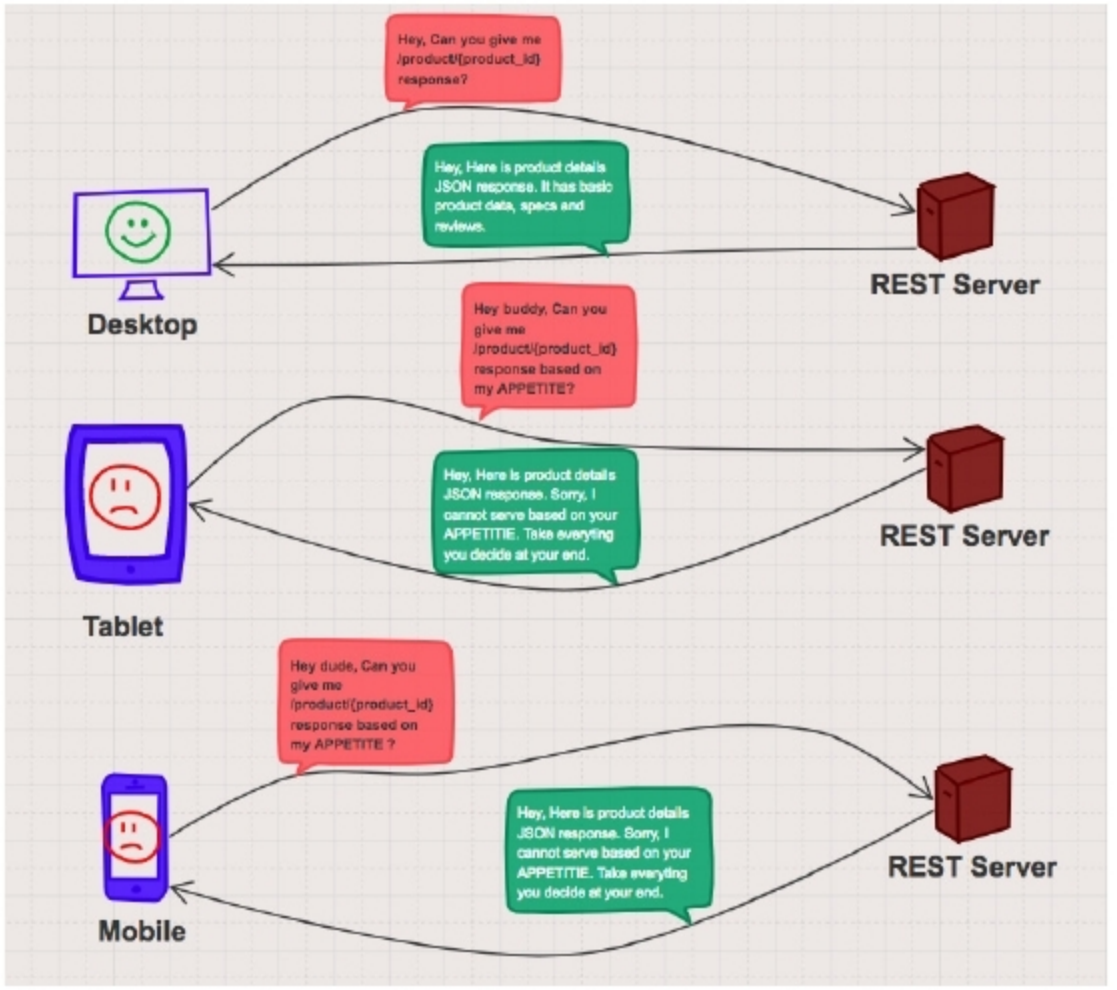

Figure 2: Over-fetching in REST

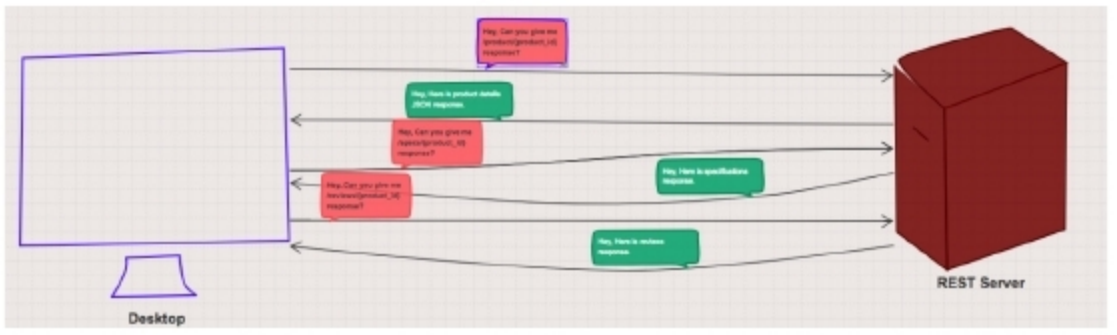

Figure 3: REST under-fetching

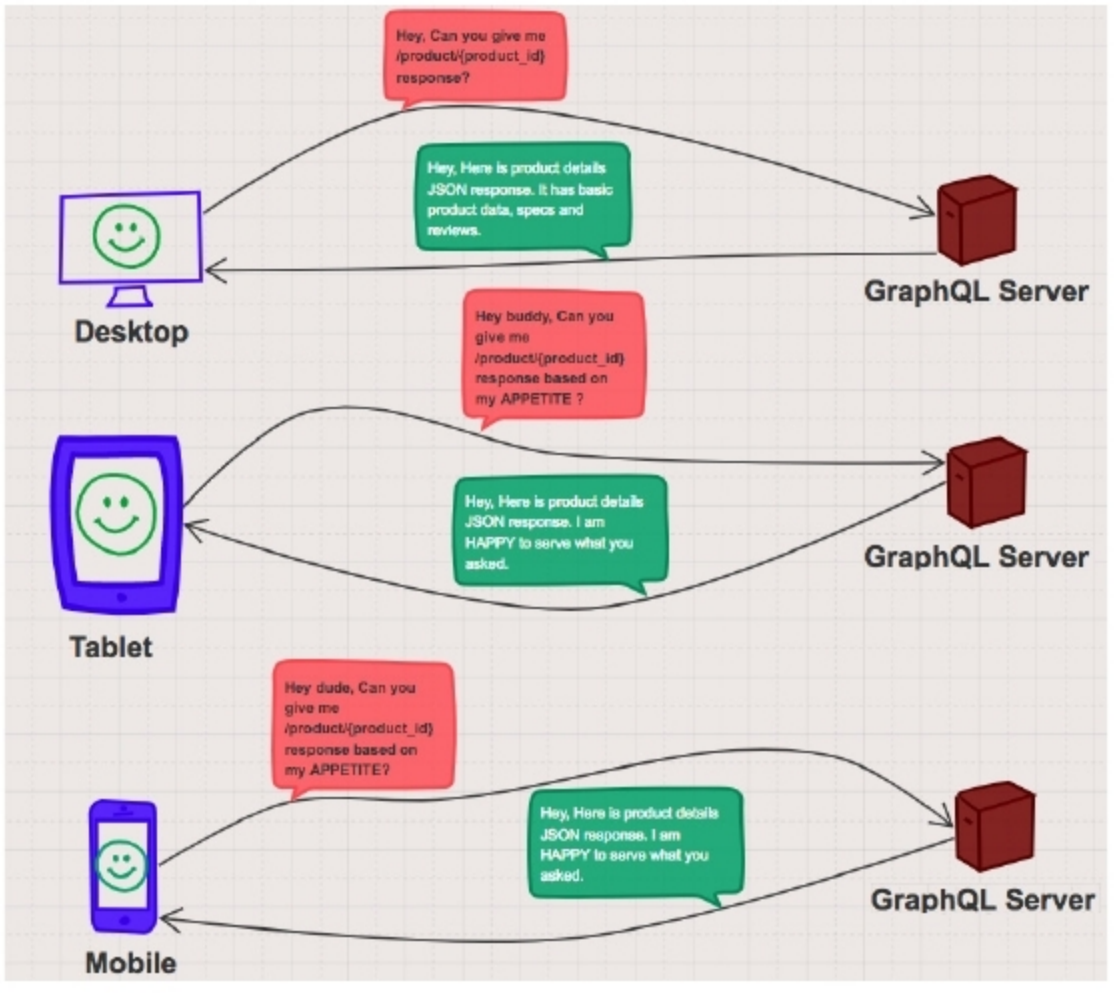

Figure 4: GraphQL

interfaces don't require the entire product JSON as Web does, even then the product detail API gives the entire product data. It is evident that clients don't have control over the data they want from the server. This is called over-fetching. The pictorial representation of the REST over-fetching issue is shown in Figure 2.

We can solve the over-fetching issue with various approaches. The straightforward approach is to maintain different APIs for thick and thin clients. Though this design solves the over-fetching issue, there are other problems like code maintenance, implementation of enhancements across different APIs, deployment of thick/thin client APIs, more compute, more manpower, etc, which increases the cost of the project. The other approach is having middleware to intercept the client request. Based on the client request, we can filter the response to return. But this adds an additional layer to the application, which has the same issues as the previous approach.

Now let us discuss the second issue with REST, which is called under-fetching. To avoid over-fetching, we can create granular APIs so that clients can make API calls for whatever data they require. Let us consider the product detail page for the Web. It has product information, specifications and reviews to display. Now, to render the product detail page, the client is not going to get data in a single API call. So, clients need to make multiple API calls (like the basic product API, the specification API, the reviews API, etc) to cater to their data requirement. This design has performance issues because of the increased number of round trips to the back-end server and API gateway. The other issue is the requirement of more compute power and networks to cater to the rise in the number of requests to serve. A pictorial representation of under-fetching is shown in Figure 3.

Let us look at the third issue with REST, which is, APIs with new versions. Any API evolves as business needs change with time. As per our customers' needs, we might need to add data attributes (in most cases we won't remove data attributes as we need to have backward compatibility) to existing APIs. When we make any changes